Exercício 4.3: Depuração: três rollouts quebrados

Aluno: Luiz Brito · **Data:** 19/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind v0.32.0 com 3 nós, Kubernetes v1.36.1, Calico v3.33.0, namespace `store-dev`.
- Base saudável em `01-deployment.yaml`: Deployment `store-api` com 3 réplicas, `RollingUpdate` com `maxSurge: 1` e `maxUnavailable: 0`, readiness em `/ready`, liveness em `/health`, um Pod por nó via `topologySpreadConstraints`, e o Service `store-api`.
- Pod cliente `curl` (`curlimages/curl:8.10.1`) para testar o Service.
- Manifestos quebrados: `02-readiness-broken.yaml` (cenário B) e `03-pdb-broken.yaml` (cenário C). Corrigidos: `01-deployment.yaml` e `04-pdb-fixed.yaml`.
- Colunas vazias `NOMINATED NODE` e `READINESS GATES` omitidas nas saídas `-o wide`.

Manifesto base

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
  namespace: store-dev
  labels:
    app: store-api
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
```

```yaml
    matchLabels:
      app: store-api
  template:
    metadata:
      labels:
        app: store-api
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: store-api
      containers:
        - name: store-api
          image: store-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: http
            periodSeconds: 10
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: store-api
  namespace: store-dev
spec:
  selector:
    app: store-api
  ports:
```

```
    - port: 80
      targetPort: http
```

Laudo A: imagem inexistente

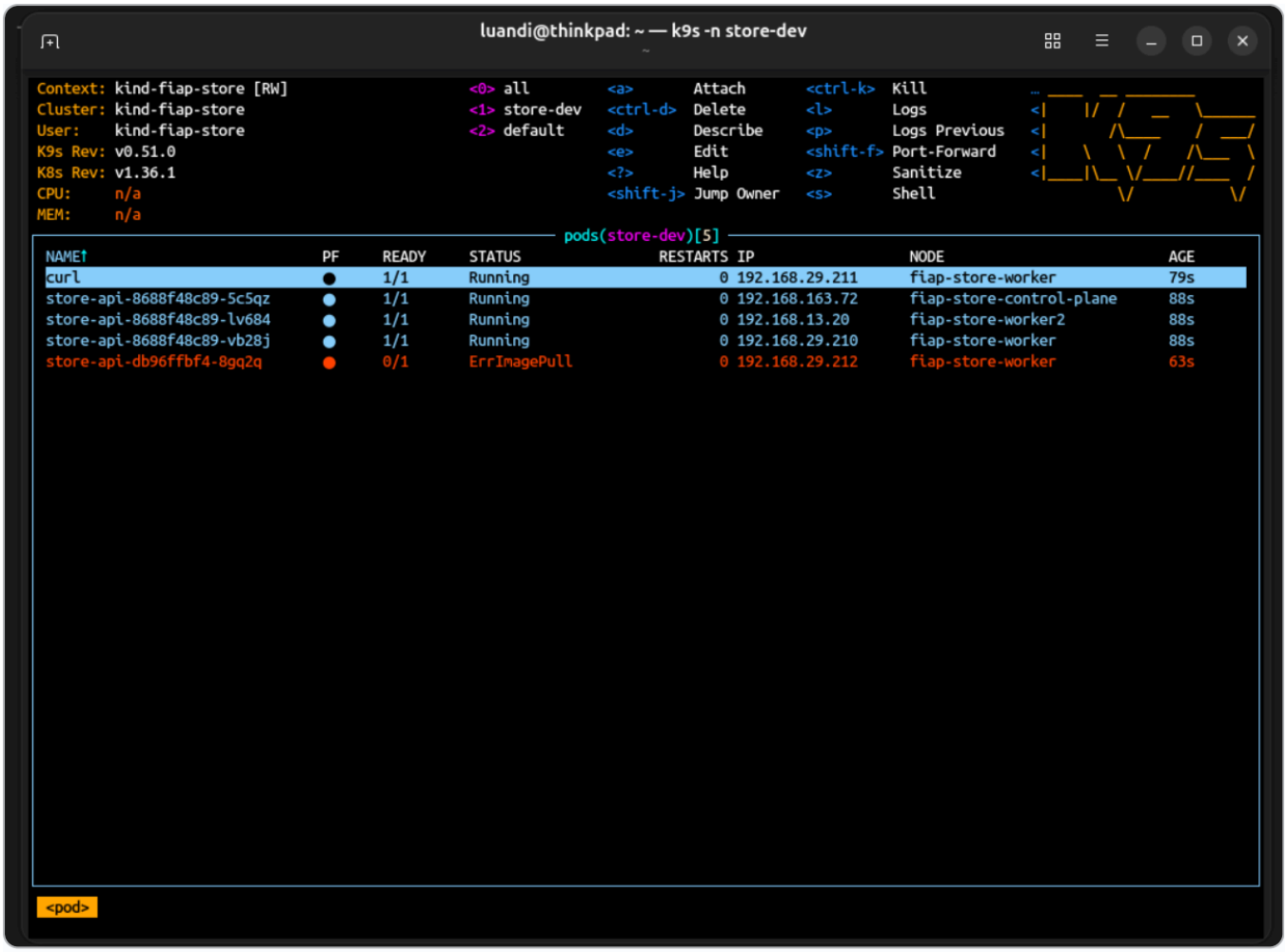
Sintoma. `rollout status` trava em "1 out of 3 new replicas have been updated" e estoura o timeout.

```
$ kubectl -n store-dev set image deployment/store-api store-api=store-api:9.9
deployment.apps/store-api image updated
$ kubectl -n store-dev rollout status deployment/store-api --timeout=45s
Waiting for deployment "store-api" rollout to finish: 1 out of 3 new replicas have been updated...
error: timed out waiting for the condition

$ kubectl -n store-dev get pods -o wide
NAME                         READY   STATUS             RESTARTS   AGE   IP               NODE
store-api-8688f48c89-5c5qz   1/1     Running            0          71s   192.168.163.72   fiap-store-
control-plane
store-api-8688f48c89-lv684   1/1     Running            0          71s   192.168.13.20    fiap-store-
worker2
store-api-8688f48c89-vb28j   1/1     Running            0          71s   192.168.29.210   fiap-store-
worker
store-api-db96ffbf4-8gq2q    0/1     ImagePullBackOff   0          46s   192.168.29.212   fiap-store-
worker

$ kubectl -n store-dev get endpointslice -l kubernetes.io/service-name=store-api
NAME              ADDRESSTYPE   PORTS   ENDPOINTS                                                 AGE
store-api-tt4d6   IPv4          8000    192.168.13.20,192.168.29.210,192.168.163.72 + 1 more...   71s

$ for i in 1 2 3; do kubectl -n store-dev exec curl -- curl -s http://store-api/health; echo; done
{"status":"ok","host":"store-api-8688f48c89-lv684"}
{"status":"ok","host":"store-api-8688f48c89-5c5qz"}
{"status":"ok","host":"store-api-8688f48c89-vb28j"}
```



Comando que revelou a causa. O `describe` do Pod novo:

```
$ kubectl -n store-dev describe pod store-api-db96ffbf4-8gq2q | sed -n '/Events/,$p'
Events:
  Type     Reason     Age                From               Message
  ----     ------     ----               ----               -------
  Normal   Scheduled  47s                default-scheduler  Successfully assigned store-dev/store-
api-db96ffbf4-8gq2q to fiap-store-worker
  Normal   BackOff    20s (x2 over 44s)  kubelet            spec.containers{store-api}: Back-off
pulling image "store-api:9.9"
  Warning  Failed     20s (x2 over 44s)  kubelet            spec.containers{store-api}: Error:
ImagePullBackOff
  Normal   Pulling    9s (x3 over 46s)   kubelet            spec.containers{store-api}: Pulling image
"store-api:9.9"
  Warning  Failed     9s (x3 over 45s)   kubelet            spec.containers{store-api}: Failed to
pull image "store-api:9.9": failed to pull and unpack image "docker.io/library/store-api:9.9": failed
to resolve reference "docker.io/library/store-api:9.9": pull access denied, repository does not exist
or may require authorization: server message: insufficient_scope: authorization failed
  Warning  Failed     9s (x3 over 45s)   kubelet            spec.containers{store-api}: Error:
ErrImagePull
```

```
$ kubectl -n store-dev get endpointslice -l kubernetes.io/service-name=store-api -o jsonpath='{range
.items[*].endpoints[*]}{.targetRef.name}{"  ready="}{.conditions.ready}{"\n"}{end}'
store-api-8688f48c89-lv684  ready=true
store-api-8688f48c89-vb28j  ready=true
store-api-8688f48c89-5c5qz  ready=true
store-api-db96ffbf4-8gq2q  ready=false
```

Causa raiz. A tag `store-api:9.9` não existe: não está nos nós e o kubelet tenta baixar do Docker Hub, que recusa. O Pod fica em `ErrImagePull` / `ImagePullBackOff` e nunca fica Ready.

Por que os Pods antigos continuam servindo. Com `maxUnavailable: 0` o controller só pode derrubar um Pod antigo depois que um novo fica disponível. Como o novo nunca fica, os 3 antigos ficam intactos e continuam como endpoints prontos (`ready=true`); o Pod quebrado entra no EndpointSlice com `ready=false` e não recebe tráfego. As 3 chamadas pelo Service responderam 200. É a virtude do RollingUpdate conservador: um deploy ruim trava, mas não derruba o serviço. Se o problema não for corrigido, depois de `progressDeadlineSeconds` (600s por padrão) o Deployment marca `Progressing=False` com `ProgressDeadlineExceeded`.

Correção.

```
$ kubectl -n store-dev rollout undo deployment/store-api
Warning: resource deployments/store-api was previously managed with 'kubectl apply'. Rolling back
will not update the kubectl.kubernetes.io/last-applied-configuration annotation, which may cause
unexpected behavior on future 'kubectl apply' operations. Consider using 'kubectl apply' with your
previous configuration file instead.
deployment.apps/store-api rolled back
$ kubectl -n store-dev rollout status deployment/store-api
deployment "store-api" successfully rolled out

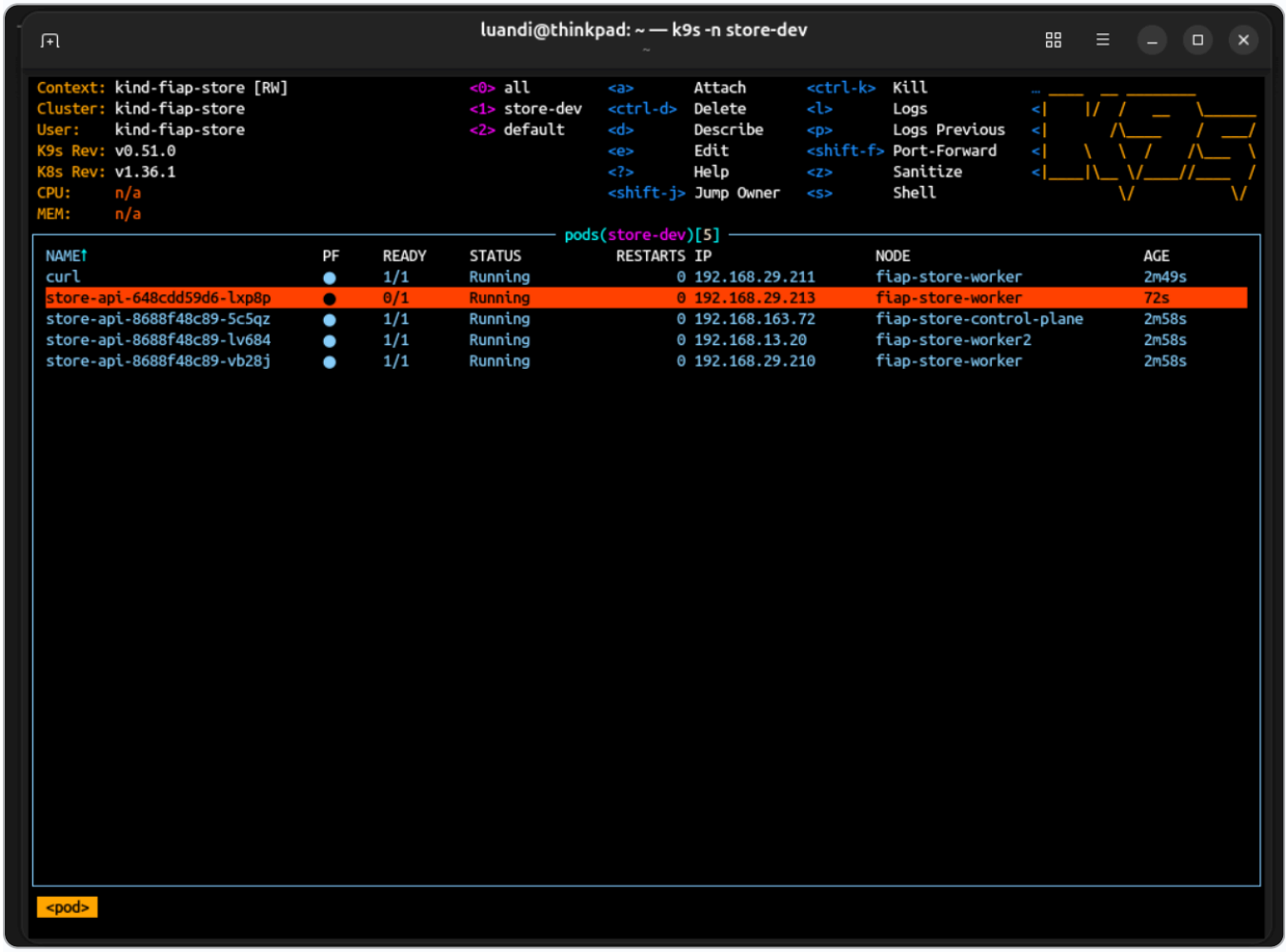
$ kubectl -n store-dev get deploy store-api
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
store-api   3/3     3            3           106s
```

Laudo B: readiness quebrada

Sintoma. Rollout trava de novo, o Pod novo está `Running` mas `0/1` e com 0 restarts.

```
$ kubectl apply -f 02-readiness-broken.yaml
deployment.apps/store-api configured
$ kubectl -n store-dev rollout status deployment/store-api --timeout=45s
Waiting for deployment "store-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "store-api" rollout to finish: 1 out of 3 new replicas have been updated...
error: timed out waiting for the condition

$ kubectl -n store-dev get pods -l app=store-api -o wide
NAME                         READY   STATUS    RESTARTS   AGE     IP               NODE
store-api-648cdd59d6-lxp8p   0/1     Running   0          45s     192.168.29.213   fiap-store-worker
store-api-8688f48c89-5c5qz   1/1     Running   0          2m31s   192.168.163.72   fiap-store-
control-plane
store-api-8688f48c89-lv684   1/1     Running   0          2m31s   192.168.13.20    fiap-store-worker2
store-api-8688f48c89-vb28j   1/1     Running   0          2m31s   192.168.29.210   fiap-store-worker
```



Comando que revelou a causa.

```
$ kubectl -n store-dev describe pod store-api-648cdd59d6-lxp8p | sed -n '/Events/,$p'
Events:
  Type     Reason     Age                From               Message
  ----     ------     ----               ----               -------
  Normal   Scheduled  45s                default-scheduler  Successfully assigned store-dev/store-
api-648cdd59d6-lxp8p to fiap-store-worker
  Normal   Pulled     44s                kubelet            spec.containers{store-api}: Container
image "store-api:1.0" already present on machine and can be accessed by the pod
  Normal   Created    44s                kubelet            spec.containers{store-api}: Container
created
  Normal   Started    44s                kubelet            spec.containers{store-api}: Container
started
  Warning  Unhealthy  44s (x2 over 44s)  kubelet            spec.containers{store-api}: Readiness
probe failed: Get "http://192.168.29.213:8000/pronto": dial tcp 192.168.29.213:8000: connect:
connection refused
  Warning  Unhealthy  4s (x8 over 39s)   kubelet            spec.containers{store-api}: Readiness
probe failed: HTTP probe failed with statuscode: 404

$ kubectl -n store-dev get endpointslice ...ready (mesmo jsonpath do cenário A)
store-api-8688f48c89-lv684  ready=true
store-api-8688f48c89-vb28j  ready=true
store-api-8688f48c89-5c5qz  ready=true
store-api-648cdd59d6-lxp8p  ready=false
```

```
luandi@thinkpad: ~ — k9s -n store-dev

Context: kind-fiap-store [RW]          <c>       Copy              <f> Toggle FullScreen
Cluster: kind-fiap-store               <e>       Edit
User:    kind-fiap-store               <n>       Next Match
K9s Rev: v0.51.0                       <shift-n> Prev Match
K8s Rev: v1.36.1                       <r>       Toggle Auto-Refresh
CPU:     n/a                           <x>       Toggle Decode
MEM:     n/a

                    Describe(store-dev/store-api-648cdd59d6-lxp8p)

  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       False
  ContainersReady             False
  PodScheduled                True
Volumes:
  kube-api-access-gf7xp:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    Burstable
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Topology Spread Constraints:  kubernetes.io/hostname:DoNotSchedule when max skew 1 is exceeded for selector app=store-api
Events:
  Type     Reason     Age                From               Message
  ----     ------     ----               ----               -------
  Normal   Scheduled  77s                default-scheduler  Successfully assigned store-dev/store-api-648cdd59d6-lxp8p to fiap-store-worker
  Normal   Pulled     76s                kubelet            spec.containers{store-api}: Container image "store-api:1.0" already present on
machine and can be accessed by the pod
  Normal   Created    76s                kubelet            spec.containers{store-api}: Container created
  Normal   Started    76s                kubelet            spec.containers{store-api}: Container started
  Warning  Unhealthy  76s (x2 over 76s)  kubelet            spec.containers{store-api}: Readiness probe failed: Get "http://192.168.29.213:
8000/pronto": dial tcp 192.168.29.213:8000: connect: connection refused
  Warning  Unhealthy  4s (x15 over 71s)  kubelet            spec.containers{store-api}: Readiness probe failed: HTTP probe failed with stat
uscode: 404

<pod> <describe>
```

Causa raiz. A `readinessProbe` aponta para `/pronto`, rota que a FastAPI não tem (404). O primeiro `connection refused` é só o uvicorn subindo; depois todas as tentativas dão 404.

Diferença se o defeito fosse na liveness. Readiness falhando só tira o Pod dos endpoints do Service: o container segue vivo, sem restart. Liveness falhando faz o kubelet **matar e reiniciar** o container a cada `failureThreshold` falhas; o contador RESTARTS sobe e o Pod entra em `CrashLoopBackOff` com backoff crescente. Nos dois casos o rollout trava por causa do `maxUnavailable: 0`, mas a liveness quebrada gasta CPU reiniciando e polui os eventos.

Correção. Voltar a readiness para `/ready` reaplicando o manifesto base:

```
$ kubectl apply -f 01-deployment.yaml
deployment.apps/store-api configured
service/store-api unchanged
$ kubectl -n store-dev rollout status deployment/store-api
deployment "store-api" successfully rolled out

$ kubectl -n store-dev get deploy store-api
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
store-api   3/3     3            3           3m26s
```

Laudo C: PDB bloqueando drain

Manifesto quebrado.

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: store-api
  namespace: store-dev
spec:
  minAvailable: 3
  selector:
    matchLabels:
      app: store-api
```

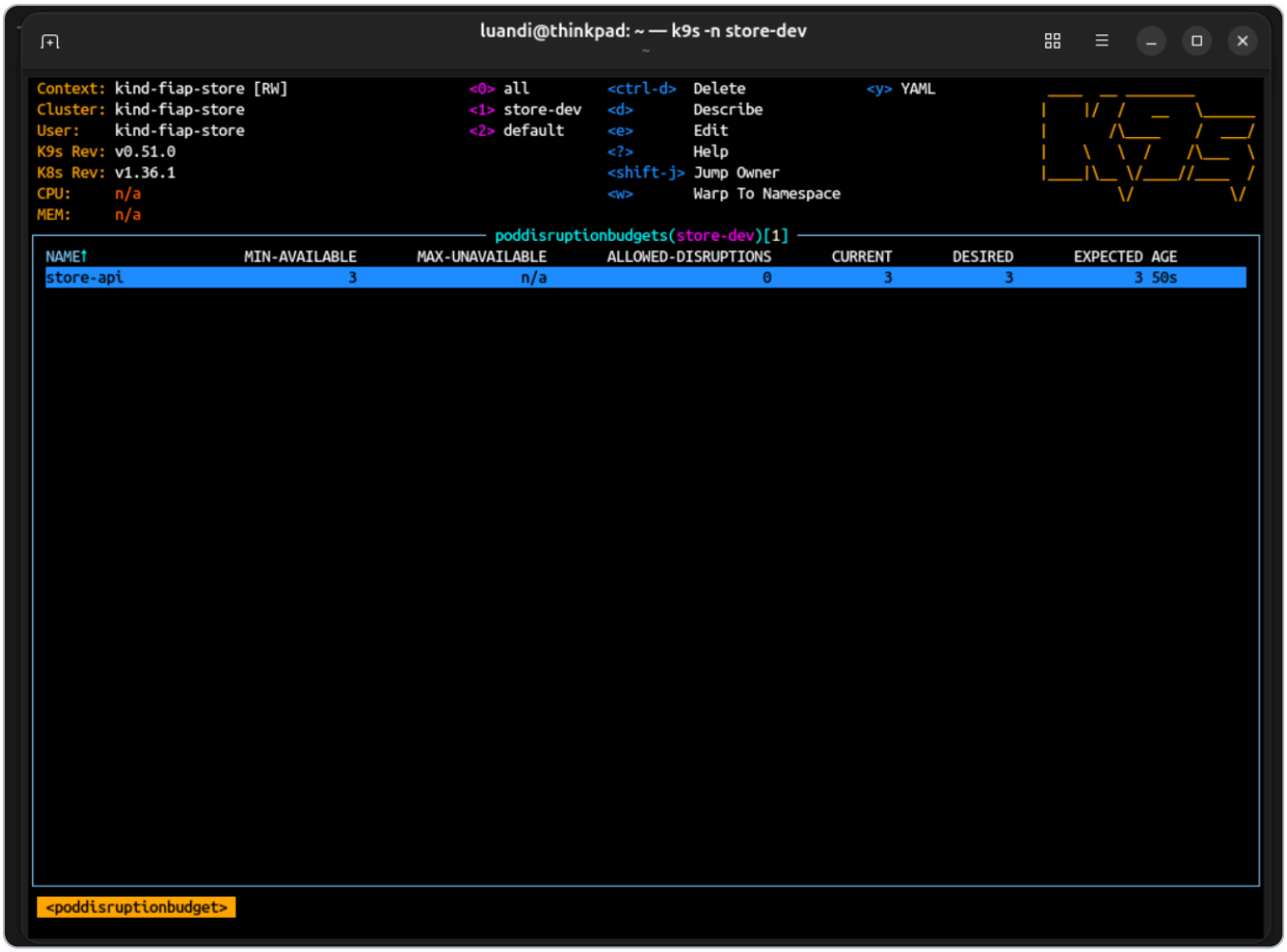
Sintoma e comando de diagnóstico. O drain fica em loop tentando despejar o Pod da `store-api` até o timeout (as linhas repetidas a cada 5s foram omitidas):

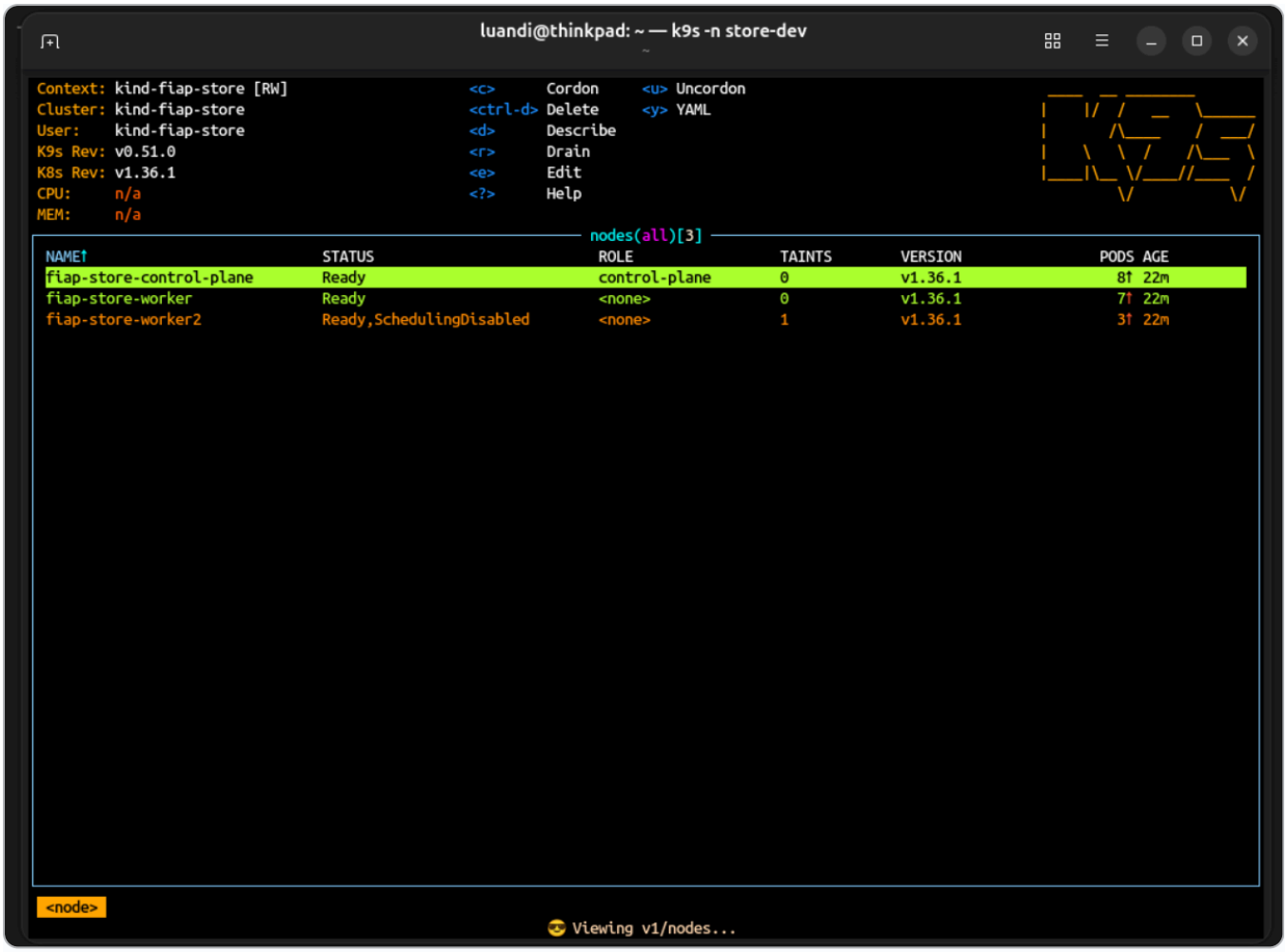
```
$ kubectl apply -f 03-pdb-broken.yaml
poddisruptionbudget.policy/store-api created

$ kubectl -n store-dev get pdb store-api
NAME        MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
store-api   3               N/A               0                     3s

$ kubectl -n store-dev get pods -l app=store-api -o wide
NAME                         STATUS    NODE
store-api-8688f48c89-5c5qz   Running   fiap-store-control-plane
store-api-8688f48c89-lv684   Running   fiap-store-worker2
store-api-8688f48c89-vb28j   Running   fiap-store-worker

$ kubectl drain fiap-store-worker2 --ignore-daemonsets --timeout=30s
node/fiap-store-worker2 cordoned
Warning: ignoring DaemonSet-managed Pods: kube-system/calico-node-4xgtf, kube-system/kube-proxy-kq8kr
evicting pod store-dev/store-api-8688f48c89-lv684
evicting pod kube-system/coredns-589f44dc88-c6q75
evicting pod kube-system/coredns-589f44dc88-7lns6
evicting pod kube-system/calico-kube-controllers-6ffffbccff-d8njd
evicting pod local-path-storage/local-path-provisioner-855c7b7774-hkqtl
error when evicting pods/"store-api-8688f48c89-lv684" -n "store-dev" (will retry after 5s): Cannot
evict pod as it would violate the pod's disruption budget.
pod/local-path-provisioner-855c7b7774-hkqtl evicted
pod/calico-kube-controllers-6ffffbccff-d8njd evicted
pod/coredns-589f44dc88-c6q75 evicted
pod/coredns-589f44dc88-7lns6 evicted
There are pending pods in node "fiap-store-worker2" when an error occurred: error when evicting
pods/"store-api-8688f48c89-lv684" -n "store-dev": global timeout reached: 30s
pod/store-api-8688f48c89-lv684
error: unable to drain node "fiap-store-worker2" due to error: error when evicting pods/"store-api-
8688f48c89-lv684" -n "store-dev": global timeout reached: 30s, continuing command...
There are pending nodes to be drained:
 fiap-store-worker2
error when evicting pods/"store-api-8688f48c89-lv684" -n "store-dev": global timeout reached: 30s
```


luandi@thinkpad: ~ — k9s -n store-dev
Context: kind-fiap-store [RW]
Cluster: kind-fiap-store
User: kind-fiap-store
K9s Rev: v0.51.0
K8s Rev: v1.36.1
CPU: n/a
MEM: n/a
<0> all
<1> store-dev
<2> default
<ctrl-d> Delete
<d> Describe
<e> Edit
<?> Help
<shift-j> Jump Owner
<w> Warp To Namespace
<y> YAML
poddisruptionbudgets(store-dev)[1]
NAME↑ MIN-AVAILABLE MAX-UNAVAILABLE ALLOWED-DISRUPTIONS CURRENT DESIRED EXPECTED AGE
store-api 3 n/a 0 3 3 3 50s
<poddisruptionbudget>



Causa raiz. O drain usa a Eviction API, que respeita o PDB. Com `minAvailable: 3` e 3 réplicas, `ALLOWED DISRUPTIONS = 3 - 3 = 0`: nenhum Pod pode sair voluntariamente, então nenhum nó com Pod da `store-api` pode ser drenado. O drain cordona o nó e fica preso.

Maior minAvailable que ainda permite drenar. Com 3 réplicas bem distribuídas (1 por nó), drenar um nó tira 1 Pod. Precisa de `ALLOWED DISRUPTIONS ≥ 1`, logo `minAvailable ≤ 3 - 1 = 2`. O maior valor é **2** (equivalente a `maxUnavailable: 1`). Como o PDB é avaliado sobre Pods **prontos**, o próximo nó só pode ser drenado depois que o substituto ficar Ready.

Correção.

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: store-api
  namespace: store-dev
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: store-api
```

```
$ kubectl apply -f 04-pdb-fixed.yaml
poddisruptionbudget.policy/store-api configured

$ kubectl -n store-dev get pdb store-api
NAME        MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
store-api   2               N/A               1                     79s

$ kubectl drain fiap-store-worker2 --ignore-daemonsets --timeout=120s
node/fiap-store-worker2 already cordoned
Warning: ignoring DaemonSet-managed Pods: kube-system/calico-node-4xgtf, kube-system/kube-proxy-kq8kr
evicting pod store-dev/store-api-8688f48c89-lv684
pod/store-api-8688f48c89-lv684 evicted
node/fiap-store-worker2 drained

$ kubectl -n store-dev get pods -l app=store-api -o wide
NAME                         STATUS    NODE
store-api-8688f48c89-5c5qz   Running   fiap-store-control-plane
store-api-8688f48c89-kj2xh   Pending   <none>
store-api-8688f48c89-vb28j   Running   fiap-store-worker

$ kubectl -n store-dev get deploy store-api
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
store-api   2/3     3            2           4m47s
```

O drain concluiu, mas o substituto ficou Pending. Investigando:

```
$ kubectl -n store-dev describe pod/store-api-8688f48c89-kj2xh | sed -n '/Events/,$p'
Events:
  Type     Reason            Age                From               Message
  ----     ------            ----               ----               -------
  Warning  FailedScheduling  11s (x2 over 11s)  default-scheduler  0/3 nodes are available: 1 node(s)
were unschedulable, 2 node(s) didn't match pod topology spread constraints. no new claims to
deallocate, preemption: 0/3 nodes are available: 1 Preemption is not helpful for scheduling, 2 No
preemption victims found for incoming pod.

$ kubectl uncordon fiap-store-worker2
node/fiap-store-worker2 uncordoned

$ kubectl -n store-dev get pods -l app=store-api -o wide
NAME                         STATUS    NODE
store-api-8688f48c89-5c5qz   Running   fiap-store-control-plane
store-api-8688f48c89-kj2xh   Running   fiap-store-worker2
store-api-8688f48c89-vb28j   Running   fiap-store-worker

$ kubectl -n store-dev get deploy store-api
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
store-api   3/3     3            3           5m12s
```

Achado extra. O substituto não foi para outro nó por causa da minha própria `topologySpreadConstraints` (hostname, `DoNotSchedule`): o nó cordoned continua contando como domínio com 0 Pods, porque o padrão de `nodeTaintsPolicy` é `Ignore` (doc oficial). Colocar um segundo Pod em outro nó daria skew 2 e a regra impede. Durante a manutenção o serviço ficou com 2 réplicas, exatamente o que o PDB corrigido garante; ao fazer `uncordon` o Pod sobe e o Deployment volta a `3/3`. Em produção, `nodeTaintsPolicy: Honor` ou `whenUnsatisfiable: ScheduleAnyway` evitaria esse efeito.